COGNITIVE PERFORMANCE AGENT

System Design & Deployment Plan

Alexa Kafka

1. Executive Summary

The Cognitive Performance Agent combines opt-in wearable signals, device-activity context, task outcomes, and evidence-informed recommendations to help a person choose among actions such as continuing, taking a short walk, or resting. It is a decision-support and wellness system, not a diagnostic device. Its core hypothesis is empirical: multimodal context may help predict when performance is likely to degrade, and personalized interventions may improve outcomes compared with a fixed-break baseline.

The recommended first production architecture is a modular backend with asynchronous workers, a managed relational database, managed queue, object storage, and a narrowly bounded LLM layer. Start with a small number of data sources, a transparent baseline model, and explicit policy gates. Add infrastructure or model complexity only when measured performance, scale, or research needs justify it.

- **Clients:** React Native mobile app, Next.js web app, and an opt-in desktop collector for coarse laptop activity features.
- **Backend:** Python/FastAPI modular service plus asynchronous workers; managed PostgreSQL with pgvector; managed task queue.
- **Decision flow:** validate data quality → compute features and personal baselines → estimate a measurable outcome → apply deterministic policy → present a recommendation.
- **LLM boundary:** generate grounded explanations and coordinate typed actions; never make final safety, medical, or access-control decisions.
- **Evaluation:** compare personalized interventions with a fixed-break baseline using task accuracy, task/recovery time, adherence, usability, and safety measures.

2. Product Scope and Design Principles

2.1 In-scope capabilities

- Ingest user-authorized health and activity observations from supported wearable/mobile-health providers.
- Collect privacy-preserving laptop context, such as app-category switches and idle intervals, with explicit opt-in.
- Optionally incorporate CGM trends when a supported integration and explicit consent are available.
- Estimate a defined near-term outcome (for example, probability that task accuracy will fall below a pre-registered threshold) and recommend an allowed action.
- Record whether the recommendation was shown, accepted, snoozed, completed, or ignored, and connect it to later task outcomes.
- Support mobile and web experiences, data-source management, consent, history, feedback, and account-data deletion.

2.2 Non-goals for the first release

- Diagnosing, treating, or claiming to prevent a medical condition.
- Capturing keystrokes, screenshots, document contents, or message contents as a default behavior.

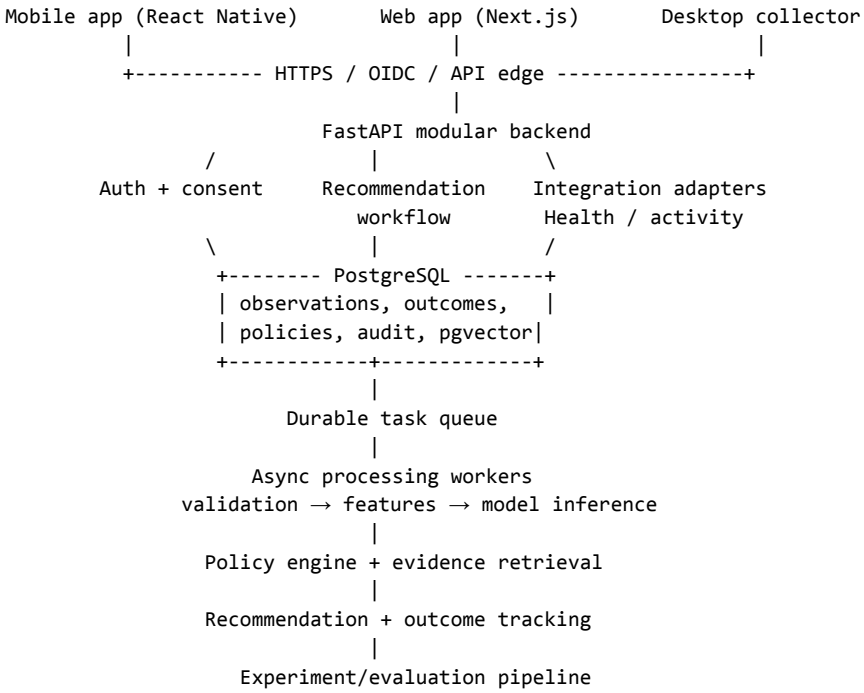
- Autonomous actions with significant consequences; the system should recommend rather than silently alter schedules or device settings.
- A complex microservice mesh, streaming platform, or custom foundation model before a demonstrated need.

2.3 Principles

- Quality before intelligence: stale, sparse, contradictory, or unsynchronized inputs must be visible to the decision policy.
- Deterministic authorization: models and LLMs may propose; code-based policy decides what is permitted.
- Personalization with uncertainty: compare against an individual baseline, expose uncertainty, and abstain when evidence is weak.
- Privacy by design: collect the least sensitive representation that supports the experiment.
- Scientific accountability: preserve model, policy, prompt, experiment, and data provenance for every recommendation.
- Graceful degradation: a missing wearable or unavailable LLM must not break core app functionality.

3. High-Level Architecture

The architecture separates client experience, ingestion, feature/model processing, recommendation policy, evidence retrieval, and evaluation. The initial deployment can be one modular API application plus worker processes rather than many independently deployed services. Logical boundaries should exist in code even where deployment units are shared.



4. Client Applications and Data Collection

4.1 Mobile application — React Native + TypeScript

React Native supports a shared TypeScript product surface while retaining native modules for platform health APIs and permission flows. Use native integrations where platform semantics matter; do not assume that a cross-platform abstraction removes OS background-execution limits.

- Core screens: onboarding and consent; data-source connections; current recommendation and rationale; action feedback; task session/outcome entry; history; privacy controls; account export/deletion.
- iOS: integrate HealthKit with the minimum requested read permissions. Apple's HealthKit authorization model is permission-based and does not guarantee that the app can infer every denied permission from an empty result. [2]
- Android: integrate Health Connect for supported data types and permissions. Verify each metric's availability, permission, and platform/version constraints. [3]
- Represent unavailable, denied, stale, and sync-failed data as different states rather than silently treating all as zero.

4.2 Web application — Next.js + TypeScript

Use Next.js for the dashboard, experiment administration, data-source status, consent settings, and detailed history. The web app is also a practical surface for researcher workflows and responsive desktop use. Keep privileged integrations and secrets server-side; the browser should receive only user-authorized, scoped data.

4.3 Desktop activity collector

A small, opt-in desktop collector can calculate coarse local features such as app-category switches, active/idle intervals, and session duration. The collector should transmit summaries rather than raw activity traces wherever feasible.

- Default deny: no keystrokes, screenshots, clipboard contents, window titles containing sensitive information, URLs, or document contents.
- Make collection visible and pausable; explain the exact feature categories collected and provide a simple uninstall/disable path.
- Batch and retry uploads; attach event time, local timezone, collector version, and data-quality metadata.
- Avoid platform-specific background-service assumptions until the target OS and distribution model are selected.

4.4 Wearable and optional CGM integrations

Build source adapters behind a common interface. Start with one high-value wearable route through HealthKit or Health Connect and add direct provider APIs only when needed. Direct APIs may require vendor approval, scopes, rate-limit handling, and refresh-token lifecycle management. CGM is optional and should be added only after consent, source access, and its research value are established.

5. Backend, Storage, and Infrastructure Choices

Decision	Recommendation	Reasoning
API framework	Python + FastAPI	Strong fit for ML/data tooling, typed request/response schemas, async I/O, and rapid iteration. Keep API handlers thin; move long-running work to workers.
Backend shape	Modular monolith + workers	Keeps early deployment and debugging simple while preserving module boundaries. Split services only when ownership, scaling, or reliability data justify it.
Primary database	Managed PostgreSQL	Transactional integrity for consent, observations, recommendations, experiments, and audit records; mature backup, migration, and access-control tooling.
Vector retrieval	pgvector in PostgreSQL	Avoids operating a separate vector database at early scale and keeps evidence metadata close to source

		records. Start with exact search; add approximate HNSW indexing only when latency/volume justify the recall/maintenance trade-off. [4]
Queue	Managed durable queue; Redis/Celery if appropriate	Decouples ingestion from feature computation and model calls. Prefer managed durability and retry/dead-letter semantics; use Redis/Celery where operational familiarity and throughput fit.
Object storage	Managed object store	Suitable for versioned model artifacts, approved research documents, and larger exports without bloating relational tables.
Identity	Managed OIDC/OAuth provider	Reduces custom authentication risk; supports short-lived sessions, account lifecycle, and standard authorization patterns.
LLM access	Provider adapter + structured outputs	Allows provider/model changes and enforces typed response contracts. Structured outputs reduce parsing ambiguity but do not replace policy validation. [5]
Containers	Docker on a managed container platform	Reproducible builds and straightforward deployment without taking on Kubernetes operations prematurely.
Infrastructure as code	Terraform	Makes environments reproducible, reviewable, and recoverable; state and secrets require access controls.
Observability	OpenTelemetry instrumentation	Vendor-neutral traces, metrics, and logs help diagnose queue delays, integrations, model latency, and end-to-end recommendation paths. [6]

5.1 Environment layout

- Separate development, staging, and production projects/accounts or equivalent isolation boundaries.
- Keep the database, queue, and internal worker endpoints private; expose only the public API edge.
- Use separate service identities and secrets per environment. Never copy production health data into development.
- Use encrypted backups and test restoration; a backup that has never been restored is not a verified recovery plan.

5.2 Why not start with Kubernetes or many microservices?

The system's largest early risks are data quality, consent, experimental validity, and recommendation safety—not raw request throughput. A modular monolith and managed services reduce operational burden, speed iteration, and make transactions easier to reason about. Extract a service only when there is a measured need for independent scaling, failure isolation, deployment cadence, or team ownership.

6. Data Contracts and Processing Pipeline

6.1 Canonical observation schema

Field	Purpose

user_id	Internal subject identifier; do not use email as the primary join key.
source	Normalized source identifier, such as HealthKit, Health Connect, wearable API, or desktop collector.
metric	Canonical metric name, e.g. resting_heart_rate, sleep_duration, app_switch_count.
value / unit	Numeric or typed value with explicit unit; avoid ambiguous strings.
observed_at	Time the measurement/event occurred, stored in UTC with source timezone metadata when relevant.
received_at	Time the backend received the record; supports lag and freshness analysis.
source_record_id	Provider record identifier or stable deduplication key where available.
quality_status	Validated, stale, incomplete, rejected, or another explicit quality state.
schema_version	Version of the canonical data contract.

6.2 Ingestion flow

1. Authenticate the user and validate integration scope and consent.
2. Receive or fetch provider records through a source-specific adapter.
3. Normalize timestamps, units, metric names, source IDs, and quality metadata.
4. Validate range, schema, ownership, and provider-specific constraints; quarantine invalid records without discarding diagnostic metadata.
5. Deduplicate using provider IDs or stable source/time/value keys where appropriate.
6. Persist observations and an outbox event transactionally.
7. Publish processing work; workers compute features and update freshness/coverage indicators idempotently.
8. Persist feature-window versions and model predictions with source-data references, then request policy evaluation.

6.3 Freshness, coverage, and missingness

Each source should expose last-successful-sync time, latest observation time, coverage over the target window, and a reason code for missing data. The policy engine should require source-specific freshness and minimum coverage before using a signal. A lack of data is not evidence of a normal value.

6.4 Time alignment

Store canonical timestamps in UTC and retain source timezone/context where needed. Feature windows must define their boundaries explicitly, including daylight-saving behavior. For wearable summaries, document whether values are point observations, intervals, daily summaries, or estimates generated by a vendor.

7. ML and Personalization

7.1 Define a measurable target

Avoid an unexplained ‘brain readiness’ score. Define a target that can be observed and evaluated—for example, the probability that task accuracy will fall below a pre-registered threshold in the next task interval. The precise target and threshold should be selected with the research protocol and validated against actual task outcomes, not assumed from physiological signals.

7.2 Initial modeling approach

- Begin with a transparent baseline: regularized logistic regression or a similarly interpretable model, plus a rules-only fallback.

- Use rolling personal baselines for features where within-person change is meaningful; keep population-level and personal features distinguishable.
- Train offline with participant-level and time-aware splits to reduce leakage from repeated sessions. Do not randomly split individual rows when sessions from the same person can appear in both train and test.
- Measure discrimination and calibration. If the system displays probabilities, evaluate calibration and recalibrate when appropriate. [7]
- Version data definitions, feature transformations, training code, model artifact, and evaluation report together.

7.3 Online inference and abstention

At recommendation time, construct a feature window only from eligible, sufficiently fresh data. Return a prediction, confidence/uncertainty indicators, model version, feature-window reference, and quality flags. If coverage is insufficient, the model is out of distribution, or confidence is below a defined threshold, abstain and use a neutral fallback such as 'not enough recent data to personalize a recommendation.'

7.4 Drift and release gates

- Track feature distributions, missingness, calibration, subgroup performance where ethically and statistically appropriate, and task-outcome relationships.
- Require offline evaluation and a staged rollout before replacing a model.
- Keep the previous model artifact available for rapid rollback.
- Do not use engagement or recommendation acceptance alone as evidence that the model improves performance.

8. Agent Workflow and Recommendation Policy

Component	Responsibility	Allowed behavior / boundary
Signal agent	Summarize validated sensor and activity features.	Can identify gaps, trends, and uncertainty; cannot invent missing values or override quality gates.
Evidence agent	Retrieve approved research/evidence snippets and source metadata.	Must cite provenance; retrieved text is untrusted input, not executable instructions.
Action agent	Propose a typed recommendation such as continue, walk, or rest.	Must select only from an allowlist and provide a concise rationale linked to evidence/features.
Policy engine	Authorize, modify, delay, or reject the proposed action.	Deterministic checks for consent, freshness, confidence, quiet hours, snooze, rate limits, safety constraints, and allowed action types.
Orchestrator	Run stages, apply timeouts, record versions, and handle failure.	Must support retries, cancellation, idempotency, and a non-LLM fallback.

8.1 Recommendation flow

1. Trigger on a user request, task-session event, or scheduled evaluation that the user has consented to.
2. Check consent, source freshness, coverage, and user-level rate limits.
3. Compute the feature window and obtain a versioned model prediction.
4. Retrieve relevant approved evidence, filtered by topic, population, recency, and provenance metadata.
5. Ask the LLM/provider adapter for a structured candidate action and rationale, if the workflow requires it.
6. Validate schema and semantic constraints; run the deterministic policy engine.
7. Persist the candidate, policy decision, versions, and rationale before presenting the recommendation.
8. Collect user response and intervention completion, then link later task outcomes to the experiment/session.

8.2 Failure behavior

- LLM timeout or provider outage: use a deterministic template or return no personalized recommendation.
- Evidence retrieval unavailable: either use a clearly labeled pre-approved generic message or abstain; do not fabricate citations.
- Stale or low-coverage data: explain that the system lacks enough recent information.
- Policy rejection: record a reason code and do not expose the rejected candidate as an approved action.
- Retries: use bounded exponential backoff, idempotency keys, and dead-letter handling for persistent failures.

8.3 LLM safeguards

- Use structured outputs or tool-call schemas, then validate every field server-side. [5]
- Never send raw health history or detailed app-activity traces to the LLM by default; provide only the minimum derived context required.
- Keep secrets, credentials, and system prompts out of client code and logs.
- Treat retrieved evidence and user-provided content as untrusted; isolate instructions from data and prohibit arbitrary tool execution.
- The LLM cannot diagnose, alter consent, bypass rate limits, or make final safety/access decisions.

9. Database and API Design

9.1 Core entities

Entity	Purpose
users / roles	Identity linkage and authorization context.
consents	Purpose-specific permissions, policy version, timestamp, revocation state.
device_connections	Provider, scopes, token references, sync status, last success.
observations	Canonical source observations and quality metadata.
feature_windows	Versioned aggregate features with source references and coverage.
personal_baselines	Versioned per-user reference distributions or parameters.
model_predictions	Target definition, score/probability, uncertainty, model version, feature-window ID.
recommendations	Candidate and displayed recommendation, rationale, timestamps, expiry.
policy_decisions	Allow/deny/modify result, reason codes, policy version.
intervention_events	Shown, accepted, snoozed, completed, ignored, or cancelled events.
task_sessions / task_outcomes	Task metadata, accuracy, task time, recovery time, outcome provenance.
experiment_assignments	Randomization arm, assignment timestamp, protocol version.
evidence_sources / evidence_chunks	Approved sources, excerpts, citations, versioning, embeddings.
audit_events / outbox_events	Security-relevant actions and reliable asynchronous event delivery.

9.2 API surface (illustrative)

All endpoints below are proposed interface examples, not claims about an existing implementation.

Endpoint	Purpose
POST /v1/integrations/{source}/connect	Begin source authorization or connection.

POST /v1/integrations/{source}/sync	Request an authorized sync; return job status rather than blocking on long jobs.
GET /v1/data/quality	Return per-source freshness, coverage, and reason codes.
GET /v1/readiness/latest	Return latest eligible prediction with model/version and uncertainty metadata.
POST /v1/recommendations/evaluate	Request an evaluation for a task/session or supported trigger.
GET /v1/recommendations/{id}	Read a persisted recommendation and policy status.
POST /v1/recommendations/{id}/feedback	Record accept, snooze, reject, or other supported feedback.
POST /v1/tasks/sessions	Create a task session.
POST /v1/tasks/sessions/{id}/outcomes	Attach measured outcomes.
GET /v1/history	Read authorized recommendation and outcome history.
GET /v1/evidence/{id}	Read evidence metadata and provenance when permitted.
POST /v1/account/data-deletion	Initiate an auditable deletion workflow.

9.3 Authorization and database protections

Perform authorization in application code and add PostgreSQL row-level security as defense in depth. RLS can restrict which rows a database role may read or modify, but table owners and privileged roles may bypass policies depending on configuration; test with the actual production role model. [8]

- Use tenant/user-scoped queries and deny-by-default authorization.
- Keep privileged migration and service roles separate from normal request roles.
- Use parameterized queries, strict input validation, and audit logging for consent changes, exports, and deletions.
- Do not put raw health values, provider tokens, or detailed activity traces into routine application logs.

10. Security, Privacy, and Governance

10.1 Controls

- TLS in transit and managed encryption at rest for databases, object storage, backups, and queues where supported.
- Short-lived access tokens, secure refresh-token handling, secret-manager storage, and least-privilege service identities.
- Granular consent by source and purpose; clear explanations of collection, derived features, retention, and withdrawal.
- Data minimization at collection and LLM boundaries; aggregate locally when feasible.
- Retention schedules and deletion workflows that include raw observations, features, baselines, embeddings, exports, and derived recommendation history as policy requires.
- Separate research/analytics datasets from operational identifiers; apply access controls and de-identification appropriate to the protocol.
- Incident response plan, dependency scanning, vulnerability management, backup restoration drills, and access reviews.

10.2 Wellness framing and research ethics

Keep claims aligned with what has been validated. The FDA's January 2026 General Wellness guidance is a useful primary reference when assessing low-risk wellness positioning, but it does not automatically determine the product's regulatory status. Claims, intended use, functionality, and marketing matter. [9] If the project involves human-subject research, determine applicable institutional review, consent, privacy, and data-management requirements before recruiting participants.

11. Experimentation and Evaluation

11.1 Evaluation design

- Pre-register the primary outcome and analysis plan before interpreting intervention effects.
- Compare personalized recommendations with a fixed-break baseline, as proposed in the pitch.
- Choose randomization at participant or session level based on carryover, contamination, and study practicality; document the rationale.
- Track task accuracy, task duration, recovery time, intervention completion, adverse or uncomfortable experiences, missing outcomes, and usability.
- Run modality ablations to estimate the incremental value of wearable, laptop/phone activity, and optional CGM signals.
- Report uncertainty, confidence intervals, exclusions, missingness, and subgroup limitations; avoid overstating small or exploratory results.

11.2 Instrumentation and causal caution

Recommendation acceptance is a process measure, not proof of benefit. A person may accept a recommendation without improving performance, or ignore a useful recommendation. Measure outcomes independently where feasible, and distinguish prediction quality from intervention effectiveness.

11.3 Suggested release gates

Gate	Evidence required before advancing
Data readiness	Source permissions work; timestamps/units validated; freshness and missingness visible; deduplication tested.
Model readiness	Participant/time-aware holdout evaluation; calibration and uncertainty assessed; fallback behavior tested.
Policy readiness	Allowlist, consent, quiet hours, stale-data, rate-limit, and failure-path tests pass.
Pilot readiness	Consent materials, support/contact process, incident response, and protocol review complete.
Scale-up readiness	No material safety/privacy regressions; primary outcome and operational metrics reviewed; model/policy versions reproducible.

12. Deployment and CI/CD Plan

12.1 Pipeline

9. Run formatting, linting, static type checks, unit tests, and schema/contract tests.
10. Run dependency, container, secret, and static security scans.
11. Build immutable versioned container images and publish artifact metadata.
12. Apply database migrations to a disposable test database and verify rollback/compatibility assumptions.
13. Deploy to staging; run integration, end-to-end, ingestion, policy, model evaluation, and failure-injection tests.
14. Promote the same built artifact to production after review; avoid rebuilding a different image for production.
15. Use health checks, canary/gradual rollout where supported, post-deploy smoke tests, and automated rollback triggers.
16. Track code, configuration, prompt, policy, model, and feature-schema versions independently.

12.2 Migration strategy

Use expand-and-contract migrations: add backward-compatible schema elements first, deploy code that can read/write both forms when needed, migrate data, then remove obsolete fields in a later release. Avoid a deployment that requires all API and worker instances to change at exactly the same moment.

12.3 Operational ownership

- Document service owners, escalation paths, on-call expectations, provider integration contacts, and recovery procedures.
- Maintain runbooks for queue backlog, provider outages, token expiration, database saturation, LLM outage, and data-deletion failures.
- Run scheduled restoration tests and verify that backups, keys, and infrastructure configuration can be recovered together.

13. SLOs, Monitoring, and Observability

The following are starting targets for planning, not measured commitments. Revisit them after a pilot establishes workload and user expectations.

Metric / SLO	Illustrative initial target	Why it matters
Monthly API availability	99.9%	Baseline reliability target for user-facing reads and writes.
Simple API read latency	p95 < 300 ms	Keeps dashboards and status checks responsive.
Recommendation latency	p95 < 5 s when dependencies are healthy	Bounds interactive waiting; timeouts and fallbacks still apply.
Durable ingestion	No acknowledged event lost after durable commit	Protects the experiment's data integrity.
Recommendation provenance	100% of persisted recommendations include model and policy versions	Enables debugging, audit, and reproducibility.
Recovery objectives	Set and test RPO/RTO before pilot	Makes recovery expectations explicit.

13.1 Monitor

- API latency/error rate, database connection saturation, queue depth/age, worker retries, and dead-letter counts.
- Provider sync success, token refresh failures, data freshness, coverage, duplicate rates, and invalid observations.
- Model latency, prediction distribution, calibration/drift indicators, abstention rate, and feature availability.
- LLM latency, timeout/error rate, token/cost estimates, schema validation failures, and fallback frequency.
- Recommendation outcomes, intervention completion, experiment-arm balance, missing outcomes, and deletion workflow status.
- Use OpenTelemetry traces and metrics across API, queue, worker, model, and provider calls; redact or avoid sensitive payloads. [6]

14. Scaling Strategy

Stage	Recommended approach	Scale trigger
Prototype / early pilot	One modular API, separate workers, managed PostgreSQL, managed queue, one wearable route, coarse desktop collector, simple model.	Focus on correctness, research workflow, and source reliability.
Growing pilot	Increase worker concurrency; tune DB indexes; partition high-volume observations if measured necessary; cache safe read paths.	Queue lag, DB load, or latency breaches persist under expected load.
Multiple teams / larger usage	Extract integration or inference services only where independent scaling/ownership materially helps; consider dedicated vector infrastructure only if needed.	Measured bottlenecks or deployment coupling outweigh added operational cost.

Mature product	Formal SRE practices, more comprehensive privacy governance, robust experiment platform, stronger model monitoring, and region/resilience decisions.	Validated product-market need, workload, compliance obligations, and operational maturity.

15. Illustrative Implementation Roadmap

This is an illustrative sequencing plan; actual duration depends on team size, provider access, research approvals, and the target desktop OS.

Phase	Focus	Exit criteria
Weeks 1–2	Architecture foundation, canonical observation contract, consent model, outcome definition, infrastructure skeleton.	Reviewed schemas, threat model, first deployable API, measurable primary target.
Weeks 3–5	Mobile/web foundations, first health integration, source status/quality, desktop collector prototype.	End-to-end opt-in ingestion with visible freshness and permission states.
Weeks 6–8	Feature pipeline, baseline model, personal baselines, policy engine, task/outcome capture.	Versioned predictions, tested abstention/fallback, complete event provenance.
Weeks 9–10	Evidence ingestion/retrieval, structured LLM layer, rationale display, feedback loop.	Grounded recommendations constrained by deterministic policy; LLM outage tests pass.
Weeks 11–12	Security/privacy hardening, CI/CD, observability, deletion/retention workflows, pilot readiness.	Staging gates pass, runbooks and rollback tested, protocol/consent review complete.
Pilot and iterate	Randomized comparison, modality ablations, calibration and usability refinement.	Outcome evidence supports or rejects the hypothesis; changes are versioned and reproducible.

16. Key Risks and Mitigations

Risk	Mitigation
Signals do not reliably predict task performance	Define target first; use baseline models; validate on participant/time-aware holdouts; allow abstention.
Wearable data is sparse, delayed, or inconsistent	Track source freshness/coverage, distinguish missingness from normal values, and gate recommendations.
Personalization creates false confidence	Calibrate probabilities, expose uncertainty, preserve baseline fallback, and monitor drift.
LLM fabricates evidence or oversteps	Approved evidence corpus, provenance, structured outputs, server-side validation, deterministic policy, and no arbitrary tools.
Privacy concerns reduce trust or enrollment	Granular opt-in, local aggregation, clear data inventory, minimal retention, easy pause/revoke/delete controls.
Provider API or OS constraints block continuous collection	Design for intermittent sync; verify each platform's permissions and background limits; do not promise uninterrupted capture.
Research results are biased by carryover or missing outcomes	Pre-register analysis; choose randomization level deliberately; report missingness and uncertainty.
Operational complexity slows iteration	Start with managed services and a modular monolith; add infrastructure only after measuring a bottleneck.

17. Primary Sources and Technical References

- [1] Original pitch slide / project scope
- [2] Apple — Authorizing access to health data
  https://developer.apple.com/documentation/healthkit/authorizing-access-to-health-data
- [3] Android Developers — Health Connect data types
  https://developer.android.com/health-and-fitness/health-connect/data-types
- [4] pgvector — Open-source vector similarity search for Postgres
  https://github.com/pgvector/pgvector
- [5] OpenAI API — Structured Outputs guide
  https://developers.openai.com/api/docs/guides/structured-outputs
- [6] OpenTelemetry — Documentation
  https://opentelemetry.io/docs/
- [7] scikit-learn — Probability calibration
  https://scikit-learn.org/stable/modules/calibration.html
- [8] PostgreSQL 17 — Row Security Policies
  https://www.postgresql.org/docs/17/ddl-rowsecurity.html
- [9] U.S. FDA — General Wellness: Policy for Low Risk Devices (January 2026)
  https://www.fda.gov/regulatory-information/search-fda-guidance-documents/general-wellness-policy-low-risk-devices
- [10] NIST — AI Risk Management Framework
  https://www.nist.gov/itl/ai-risk-management-framework
- [11] FastAPI documentation
  https://fastapi.tiangolo.com/
- [12] React Native documentation
  https://reactnative.dev/docs/getting-started
- [13] Next.js documentation
  https://nextjs.org/docs